SC4021 Information Retrieval

NTU BSc Computer Science — Comprehensive Textbook (0–100)

NTU CS Curriculum Textbook Series
College of Computing and Data Science

2026-08-30 — Generated for bumbsclaw/ntu-cs-textbooks

Contents

1 IR Foundations: From Need to Ranked List **1**

1.1 Intuition: A Need is Not a Query . . . 1

1.2 IR vs. Databases vs. Data Mining . . . 2

1.3 The IR Pipeline . . . 2

1.4 Tokenization, Normalization, Vocabulary . . . 3

1.5 Running Example: A 5-Document Collection . . . 4

1.6 Why Ranking Matters: A User Study Intuition . . . 4

1.7 Crawl, Index Size, and Zipf Implications . . . 4

1.8 Vocabulary Growth and Heaps' Law . . . 5

1.9 Chapter Notes . . . 5

1.10 Exercises . . . 5

2 Boolean Retrieval vs. Vector Space Model **7**

2.1 Boolean Retrieval: Terms as Sets . . . 7

2.2 Vector Space: Documents as Vectors . . . 8

2.3 Term-Document Matrix and Weighting Preview . . . 9

2.4 Boolean vs. Ranked: When Each Wins . . . 10

2.5 Weighting Preview and Normalization Justification . . . 10

2.6 Chapter Notes . . . 10

2.7 Exercises . . . 11

3 Term Weighting: TF–IDF and Beyond **12**

3.1 Intuition: Frequency and Rarity . . . 12

3.2 Term Frequency Variants . . . 13

3.3 Inverse Document Frequency . . . 13
3.4 Length Normalization . . . 14
3.5 Worked Example: 3-Document TF–IDF . . . 14
3.6 Beyond TF–IDF: BM25 Preview . . . 15
3.7 Choosing Heuristics: When to Use Which TF . . . 16
3.8 Chapter Notes . . . 16
3.9 Exercises . . . 16

4 The Inverted Index: Making Retrieval Fast 17

4.1 Why Not Scan? The Inverted Idea . . . 17
4.2 Building the Index . . . 18
4.3 Phrase and Proximity Queries . . . 19
4.4 Compression and Skips . . . 19
4.5 Complexity and Trade-offs . . . 20
4.6 Updates, Deletion, and Dynamic Indexing . . . 20
4.7 Fielded Search and Scoring at Query Time . . . 20
4.8 Chapter Notes . . . 21
4.9 Exercises . . . 21

5 Evaluation: Precision, Recall, MAP, and NDCG 22

5.1 Precision and Recall . . . 22
5.2 Average Precision and MAP . . . 23
5.3 Graded Relevance: DCG and NDCG . . . 24
5.4 Cranfield, Pooling, and Significance . . . 25
5.5 Micro vs. Macro and User-Oriented Metrics . . . 25
5.6 Chapter Notes . . . 26
5.7 Exercises . . . 26

6 Query Expansion and Relevance Feedback 27

6.1 Why Queries Fail . . . 27
6.2 Rocchio Feedback . . . 28

6.3 Pseudo-Relevance and Expansion . . . 29
6.4 Relevance Models and RM3 . . . 30
6.5 Evaluation of Expansion and Query Performance Prediction . . . 30
6.6 Chapter Notes . . . 30
6.7 Exercises . . . 31

7 Link Analysis: PageRank and HITS 32

7.1 The Web as a Graph . . . 32
7.2 PageRank: Random Surfer with Teleportation . . . 33
7.3 HITS: Hubs and Authorities . . . 34
7.4 Spam, Scale, Personalization . . . 35
7.5 Topic-Sensitive and Personalized PageRank . . . 35
7.6 Convergence, Implementation, and Matrix View . . . 36
7.7 Chapter Notes . . . 36
7.8 Exercises . . . 36

8 Learning to Rank and Neural IR 37

8.1 Ranking as Supervised Learning . . . 37
8.2 Neural IR: Dense Retrieval . . . 38
8.3 ANN and Hybrid Retrieval . . . 39
8.4 Evaluation, Data, and Pitfalls . . . 40
8.5 Training Data, Negatives, and Contrastive Learning . . . 40
8.6 From Ranker to System: Indexing, Latency, and Monitoring . . . 40
8.7 Chapter Notes . . . 41
8.8 Exercises . . . 41

Chapter 1

IR Foundations: From Need to Ranked List

Before we rank anything we must understand what a user wants, what a collection holds, and what "relevant" means.

From zero: no IR background assumed. We start from everyday search, define *information need*, *query*, *document*, and *relevance*, then formalise the IR pipeline. Pictures precede formalism.

Learning Outcomes

After this chapter you will be able to:

(L1) distinguish information need, query, and relevance and give examples;

(L2) contrast IR vs. databases (exact) vs. data mining (pattern discovery);

(L3) describe the IR pipeline: crawl → index → query → rank → evaluate;

(L4) explain tokenization/normalization choices and their effect on vocabulary;

(L5) scope a tiny IR task and judge relevance given a query.

1.1 Intuition: A Need is Not a Query

Intuition. You feel "I need papers about neural retrieval that handle long documents." That feeling is the *information need* (in your head, vague). You type `neural retrieval long documents` – that string is the *query* (what you told the system, lossy). The system returns docs; you label each *relevant* or not. The gap between need and query explains why IR is *ranked* not Boolean: the system must guess which docs best satisfy the unobserved need from the observed query.

Formal definitions. Let $\mathcal{C} = \{d_1, \ldots, d_N\}$ be a *collection* of N documents. Each document d is a sequence of terms from vocabulary $\mathcal{V}$ with $|\mathcal{V}| = V$. A *query* q is also a sequence (often short). A *relevance judgment* $rel(q, d) \in \{0, 1\}$ (binary) or $\{0, 1, 2, 3\}$ (graded) says how well d satisfies the need behind q. IR's task: given q, return a ranked list $d_{(1)}, \ldots, d_{(k)}$ that maximises user utility under rel.

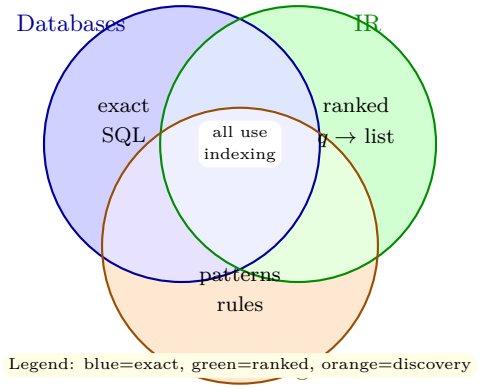
Definition 1.1 (Information Need vs. Query vs. Relevance)**.** *Information need*: the underlying task ("find evidence for X"). *Query*: the textual proxy the user provides. *Relevance*: a judged relation between query/need and document. Two users issuing the same query string can have different needs – hence personalization and evaluation difficulty.

Example 1.1 (Need $\neq$ Query)**.** Need: "How to fix a leaking tap without special tools." Query A: `fix leaking tap DIY`. Query B: `tap leak repair no tools`. Same need, different queries; a good IR system should rank the same useful doc highly for both.

1.2 IR vs. Databases vs. Data Mining

Intuition. A database answers "Give me the employee with ID 184" exactly or empty. IR answers "Find docs about retrieval" approximately – many answers, ordered by estimated usefulness. Data mining answers "What patterns hold across the collection?" (e.g., association rules). Different questions, different machinery.

Formal contrast.



Databases assume a schema and closed world; IR assumes unstructured text and open vocabulary. Yet they share indexing ideas (Ch. 4 generalises B-trees/hashing from SC2207 to inverted indexes).

1.3 The IR Pipeline

Intuition. Think of a library that must answer thousands of questions per second over millions of books without reading every book per question. It pre-processes books once (index), then per query only consults the index.

Pipeline stages. (1) **Crawl/Gather**: acquire docs. (2) **Preprocess**: tokenize, normalize (lowercase, strip punctuation, stem), build vocabulary. (3) **Index**: inverted index from terms to doc lists (Ch. 4). (4) **Query**

processing: parse q, look up postings, score (Ch. 2–3). (5) **Ranking**: sort by score. (6) **Evaluation**: measure against judgments (Ch. 5).

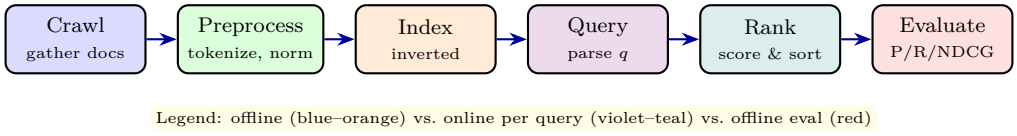


Figure 1.1: IR pipeline: most work happens offline at indexing time; queries are fast lookups.

Listing 1.1: Tiny IR pipeline: normalize, tokenize, and judge relevance.

```python
import re
def tokenize(text):
    # lowercase + keep alphanumeric words (simple normalization)
    text = text.lower()
    return re.findall(r"\b\w+\b", text)

docs = ["Neural retrieval for long documents", "Database indexing with B-trees", "Long document retrieval with transformers"]
query = "neural retrieval long documents"
q_terms = set(tokenize(query))
for i, d in enumerate(docs):
    terms = set(tokenize(d))
    overlap = len(q_terms & terms)
    print(f"Doc {i}: overlap={overlap}, terms={terms}")
# Relevance is not overlap alone: doc 2 has zero overlap => non-relevant
# Doc 0 vs 2: need human judgment for ‘‘relevant’’ (graded better)
```

1.4 Tokenization, Normalization, Vocabulary

Intuition. Should `Retrieval` equal `retrieval`? Should `don't` be `don t`, `dont`, or `do n't`? Each choice changes V and recall. Aggressive normalization shrinks V and improves recall but may conflate distinct meanings (e.g., `US` vs. `us`).

Formal. Let raw string s be mapped by normalization $n(\cdot)$ then tokenization $tok(\cdot)$ to sequence $[t_1, \ldots, t_m]$. Vocabulary $\mathcal{V} = \bigcup_{d \in \mathcal{C}} tok(n(d))$. Heaps' law: $V \approx Kn^{\beta}$ with $\beta \approx 0.5$ for English – vocabulary grows sublinearly with collection size n (tokens).

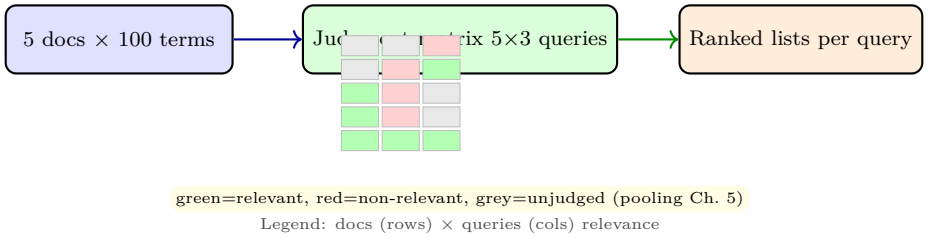


Figure 1.2: Collection → judgments → rankings: ground truth for evaluation.

1.5 Running Example: A 5-Document Collection

Consider $\mathcal{C}$: d_1='cats and dogs', d_2='cats cats cats', d_3='dogs chase cats', d_4='birds fly', d_5='cats dogs birds'. Query q='cats dogs'. Which docs are relevant? Intuitively all except d_4, but is d_2 (only cats) as relevant as d_5 (both terms)? Graded judgments help: d_5=3 (highly), d_1, d_3=2, d_2=1, d_4=0. Ranking quality will be measured against these grades in Ch. 5.

Listing 1.2: Judging relevance and the need for ranking, not just Boolean match.

```
# Graded relevance for query "cats dogs" over 5 docs
relevance = {"d1":2, "d2":1, "d3":2, "d4":0, "d5":3}
# Boolean AND would return {d1,d3,d5} (unordered) and hide that d5 >> d1
# Ranked retrieval should order: d5 (3) > d1,d3 (2) > d2 (1) > d4 (0)
ranked = sorted(relevance, key=lambda d: relevance[d], reverse=True)
print(ranked)  # ['d5', 'd1', 'd3', 'd2', 'd4']
# Lesson: Boolean is insufficient when relevance is graded.
```

1.6 Why Ranking Matters: A User Study Intuition

Intuition. Eye-tracking studies show users examine results top-down and rarely go past rank 10. A system that returns the same 10 relevant docs but in reverse order (worst first) is judged far worse, even though set recall is identical. This is why Boolean's unordered set is a poor user model and why ranked evaluation (Ch. 5) discounts by position.

Formal. Let $gain(r)$ be the utility a user gets from a relevant doc at rank r (decreasing in r). Expected utility of ranking π is $\sum_r gain(r) \cdot rel(\pi_r)$. Any retrieval model must therefore produce an ordering that maximises this sum – not just a set. This idea will reappear as NDCG's discount $\log(i+1)$ and as LambdaMART's λ weighted by NDCG swap gain.

Example. For query "NTU shuttle," two systems each retrieve 3 relevant docs. System A: ranks them at positions 1,2,3. System B: ranks them at 8,9,10. With $gain(r) = 1/\log_2(r+1)$, utilities are $1.00+0.63+0.50 = 2.13$ vs. $0.33+0.32+0.30 = 0.95$ – A is more than twice as useful. Hence "find all relevant" is not the goal; "put relevant early" is.

Remark 1.1 (IR and HCI). IR evaluation historically treated relevance as topical (aboutness). Modern evaluation adds *usefulness*, *credibility*, and *effort* – but the core pipeline need $\rightarrow q \rightarrow$ ranked list $\rightarrow$ judgment remains the abstraction we teach first.

1.7 Crawl, Index Size, and Zipf Implications

Intuition. A web crawl is never complete; the indexer must decide what to keep. Zipf's law (Ch. 3 of SC4002, and SC3020 Ch. 1) predicts most terms are rare: a term appearing once can be discarded with little recall loss but huge vocabulary savings. This is why indexes apply frequency cutoffs and stopword removal – not for linguistics, but for V and postings size.

Formal. Let collection token count be T. Number of distinct terms $V \approx KT^{\beta}$ with $\beta \approx 0.5$ (Heaps' law). Postings entries equal $\sum_t df_t$, which is $O(T)$ but with constant ≈ 0.3 due to repeats. With positions, storage is ≈ 8 bytes per posting after compression (vs. 4+4 docID+freq raw). For $T = 10^9$, expect $\sim$8 GB compressed index plus dictionary – fitting on one machine, while exhaustive scans would be $\sim$1 TB text.

Remark 1.2 (Static vs. dynamic collections). Static collections (Cranfield, TREC) are frozen for reproducible evaluation (Ch. 5). Dynamic collections (web, news) need incremental indexing (Ch. 4) and fresh judgements – evaluation becomes monitoring, not a single number. This is why industrial IR tracks live A/B metrics alongside periodic TREC-style audits.

Takeaway. The pipeline's efficiency hinges on Zipf/Heaps: rare terms dominate vocabulary but contribute little to recall; indexing focuses on the head and upper tail, compresses the rest, and evaluates on pooled head queries.

1.8 Vocabulary Growth and Heaps' Law

Intuition. As you read more docs, you keep encountering new words, but at a slowing rate – after 1M tokens you have seen most common words, so new tokens are increasingly rare names or typos. This sublinear growth governs how large a dictionary must be at scale.

Formal. Heaps' law: $V = Kn^{\beta}$ where n is collection tokens, $K \approx 30$–100, $\beta \approx 0.4$–0.6 for English. For $n = 10^6$, $V \approx 30 \cdot (10^6)^{0.5} \approx 30{,}000$; for $n = 10^9$, $V \approx 30 \cdot 31623 \approx 950{,}000$ – only $\sim$30$\times$ larger despite 1000$\times$ more tokens. Yet postings grow linearly with n (one per token occurrence), so index size scales with tokens, not vocabulary – motivating compression of postings over dictionary optimization.

1.9 Chapter Notes

IR vs. databases is a spectrum: modern systems blend keyword search with structured filters. The Cranfield paradigm (Cleverdon, 1967) – fixed collection, queries, and judgments – underlies all of Ch. 5. Tokenization choices echo NLP (SC4002 Ch. 1); IR tokenization is often simpler but must handle URLs, numbers, and case carefully at web scale.

1.10 Exercises

E1.1 **Need vs. query.** Give three different queries for the need "cheap vegetarian restaurants near campus open late" and explain why they would retrieve different results.

E1.2 **IR vs. DB vs. mining.** Fill a table: for each of (IR, DB, mining) state (a) input type, (b) query type, (c) result type, (d) typical data structure. Give one NTU example per row.

E1.3 **Normalization trade-off.** Corpus has "US", "us", "U.S." Should they map to one term or three? Discuss one benefit and one risk for each choice and propose a rule using case folding only for lowercase words.

E1.4 **Cranfield design.** You have 10,000 docs and 50 queries. You can judge only 2,000 doc-query pairs. Describe a pooling strategy (which docs to judge) and why judging random pairs would be wasteful.

E1.5 **Relevance grades.** For query "NTU shuttle bus schedule" judge 5 invented documents on a 0–3 scale (0 non-relevant ... 3 perfectly relevant). Justify each grade and propose an ideal ranking.

Chapter 2

Boolean Retrieval vs. Vector Space Model

From exact set operations to geometric ranking: why "contains the word" is not enough and how angles between vectors give order.

From zero: no retrieval model assumed. We start from sets (SC1007 review), build Boolean retrieval as set intersection/union, expose its brittle ranking, then place every document as a point in term-space and rank by cosine.

Learning Outcomes

After this chapter you will be able to:

(L1) execute Boolean queries via postings sets and explain their limits;

(L2) represent documents/queries as term vectors and build the term-document matrix;

(L3) compute cosine similarity and rank documents for a query;

(L4) compare Boolean vs. ranked retrieval and choose per task;

(L5) handle phrase and wildcard intuition via positional sets.

2.1 Boolean Retrieval: Terms as Sets

Intuition. If the query is `cats AND dogs`, a document qualifies iff its term set contains both words – a set intersection. `OR` is union, `NOT` is complement. Simple and exact, but you either match or you do not; there is no "best" among the matches, and one missing synonym kills recall.

Formal model. For term t, let $P(t) = \{d \in \mathcal{C} : t \in d\}$ be its *postings set*. Then

$$P(t_1 \text{ AND } t_2) = P(t_1) \cap P(t_2), \quad P(t_1 \text{ OR } t_2) = P(t_1) \cup P(t_2), \quad P(\text{NOT } t) = \mathcal{C} \setminus P(t). \tag{2.1}$$

With an inverted index (Ch. 4), $P(t)$ is stored sorted; intersection is a linear merge $O(|P(t_1)|+|P(t_2)|)$ or faster with skips.

Definition 2.1 (Boolean Retrieval)**.** Given query q in disjunctive/conjunctive normal form over terms, $Answer(q)$ is defined recursively via the set equations above. Result is an *unordered* set; no ranking.

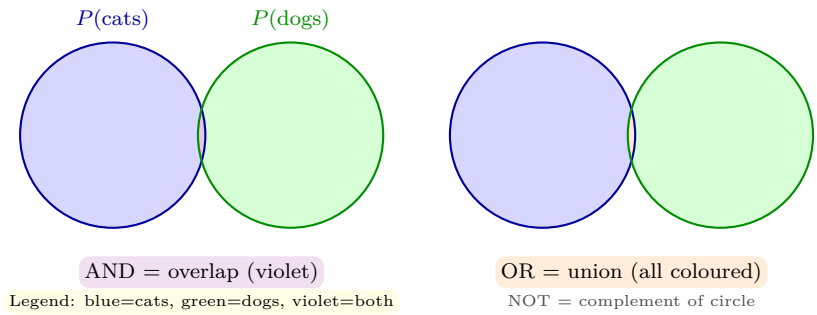


Figure 2.1: Boolean operations as set operations: AND is intersection, OR is union.

Example 2.1 (Boolean evaluation)**.** Collection: d_1={cats,dogs}, d_2={cats}, d_3={dogs,birds}, d_4={cats,dogs,birds}. Then $P(\text{cats}) = \{1,2,4\}$, $P(\text{dogs}) = \{1,3,4\}$. Query `cats AND dogs`: $\{1,2,4\} \cap \{1,3,4\} = \{1,4\}$ (unordered).
Query `cats AND NOT dogs`: $\{1,2,4\} \setminus \{1,3,4\} = \{2\}$.

Limits: no ranking among $\{1,4\}$, synonym failure (`feline` $\neq$ `cats`), and verbose queries to express "more cats than dogs matters."

2.2 Vector Space: Documents as Vectors

Intuition. Place every term as an axis in a V-dimensional space. Document d becomes a vector $\vec{d} \in \mathbb{R}^V$ where coordinate t is the weight of term t in d (for now, 0/1 or count). Query $\vec{q}$ is another vector. Documents that point in a similar direction as the query are more relevant – measured by the angle between them.

Formal. Let vocabulary $\mathcal{V} = \{t_1, \ldots, t_V\}$. Define term-document matrix $M \in \mathbb{R}^{V \times N}$ with $M_{t,d} = w(t,d)$ (weight). Column $M_{\cdot,d} = \vec{d}$ is a doc vector; query vector $\vec{q}$ has $q_t = w(t,q)$. For now w may be raw term frequency; Ch. 3 refines it to TF-IDF.

Definition 2.2 (Cosine Similarity)**.**

$$\cos(\vec{q},\vec{d}) = \frac{\vec{q}\cdot\vec{d}}{\|\vec{q}\|\,\|\vec{d}\|} = \frac{\sum_t q_t d_t}{\sqrt{\sum_t q_t^2}\sqrt{\sum_t d_t^2}} \in [-1,1], \tag{2.2}$$

or $[0,1]$ for non-negative weights. Ranking: sort d by decreasing $\cos(\vec{q},\vec{d})$.

Why cosine not Euclidean? Length normalization: a long doc that repeats terms should not automatically outrank a short, focused doc. Dividing by $\|\vec{d}\|$ removes length bias (more in Ch. 3).

Example 2.2 (Cosine by hand)**.** Vocab {cats, dogs}. $\vec{q} = (1,1)$, $\vec{d_1} = (1,1)$ (both terms), $\vec{d_2} = (3,0)$ (cats repeated). Then $\vec{q}\cdot\vec{d_1} = 2$, $\|\vec{q}\| = \sqrt{2}$, $\|\vec{d_1}\| = \sqrt{2}$, $\cos = 2/2 = 1.0$. $\vec{q}\cdot\vec{d_2} = 3$, $\|\vec{d_2}\| = 3$, $\cos = 3/(4.24) = 0.707$. So d_1 outranks d_2 despite d_2 having higher raw count of "cats" – length normalization matters.

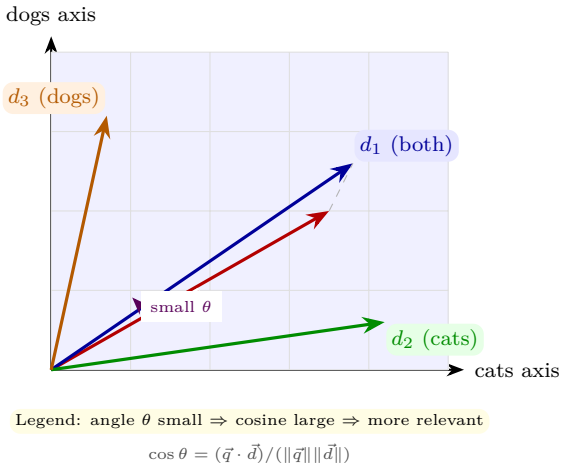


Figure 2.2: Vector space in 2-D: query and three docs. d_1 has smallest angle to q thus highest cosine.

2.3 Term-Document Matrix and Weighting Preview

The matrix view makes sparsity visible: most entries are 0 (a doc uses few of V terms). With $N = 10^6$, $V = 5 \times 10^5$, density $< 0.1\%$ – hence inverted index not dense matrix.

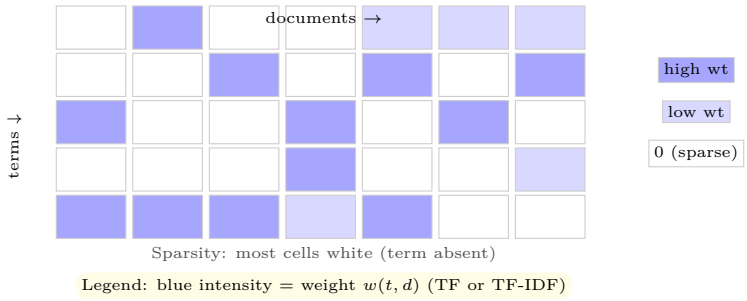


Figure 2.3: Term-document matrix heatmap: columns are doc vectors; mostly zeros.

Listing 2.1: Boolean sets and cosine ranking on the same tiny collection.

```
import numpy as np

# Boolean: postings as sets
P_cats = {1,2,4}
P_dogs = {1,3,4}
print("AND:", P_cats & P_dogs)  # {1,4}
print("OR:",  P_cats | P_dogs)  # {1,2,3,4}

# Vector space: raw TF vectors, vocab [cats, dogs]
q  = np.array([1.,1.])
d1 = np.array([1.,1.])   # both
d2 = np.array([3.,0.])   # cats cats cats
d3 = np.array([0.,1.])   # dogs
def cosine(a,b):
    return a.dot(b) / (np.linalg.norm(a)*np.linalg.norm(b))
for name, d in [("d1",d1),("d2",d2),("d3",d3)]:
    print(name, round(cosine(q,d),3))
# d1 1.0 > d2 0.707 > d3 0.707 -> tie broken by other signals
```

2.4 Boolean vs. Ranked: When Each Wins

Boolean shines for *exact* needs: "find docs containing `SC4021` AND `PageRank`" as a filter, legal discovery, or when combined with ranking (Boolean filter then rank). Ranked (vector) shines for *best-match* where gradations matter. Modern systems hybridise: use Boolean/phrase for candidate generation, then vector scoring for ranking.

Listing 2.2: Hybrid: Boolean filter then cosine re-rank.

```
# Hybrid: first Boolean AND, then rank survivors by cosine
docs = {"d1": np.array([1.,1.]), "d2": np.array([3.,0.]), "d3": np.array([0.,1.]), "d4":
    np.array([1.,0.])}
q = np.array([1.,1.])
candidates = [d for d, v in docs.items() if v[0]>0 and v[1]>0]  # AND cats AND dogs
print("candidates:", candidates)
ranked = sorted(candidates, key=lambda d: cosine(q, docs[d]), reverse=True)
print("ranked:", ranked)
```

2.5 Weighting Preview and Normalization Justification

Intuition. The vector example used raw counts. But should "the" with count 50 in a long doc outweigh "PageRank" with count 2 in a short focused doc? No. Without weighting and length normalization, long docs and frequent words dominate cosine. Ch. 3 will fix this with TF damping and IDF.

Formal tie-in. Let $\vec{d}_{raw}$ be raw-count vector. Define normalized weighted vector $\vec{d}_w = D^{-1}W\vec{d}_{raw}$ where W is diagonal with idf_t and D is length normalizer ($\|\vec{d}\|_2$ or pivoted). Then $\cos(\vec{q}_w, \vec{d}_w) = \vec{q}_w \cdot \vec{d}_w / (\|\vec{q}_w\| \|\vec{d}_w\|)$ already includes D, but additional normalizers (double-norm TF, pivoted) further correct for verbosity vs. scope: a long doc may be verbose (same topic repeated) or multi-topic (many topics, query covers one) – the correction differs.

Remark 2.1 (Boolean as extremal vector). Boolean retrieval can be seen as vector retrieval with binary weights and *threshold* $= 1$ for AND, > 0 for OR, and no normalization. Ranked retrieval generalises it by allowing graded weights and soft thresholds – one reason industry keeps both layers (Boolean *filters* + vector *rankers*).

2.6 Chapter Notes

Boolean retrieval powered early systems (STAIRS, Westlaw). Vector space (Salton, 1970s SMART system) introduced geometry to IR. The cosine remains the workhorse even inside neural models (dot products between embeddings are its descendant). Positional and fielded Boolean (title vs. body) are handled via extended postings (Ch. 4).

2.7 Exercises

E2.1 **Boolean sets.** Given $P(a) = \{1, 2, 3, 5\}$, $P(b) = \{2, 3, 4\}$, $P(c) = \{3, 5, 6\}$, compute (a) a AND b, (b) a OR b, (c) (a OR b) AND NOT c, (d) a AND b AND c. Show sorted postings merges.

E2.2 **Cosine by hand.** Vocab {IR, retrieval, database}. $\vec{q} = (1, 1, 0)$, $\vec{d_1} = (2, 1, 0)$, $\vec{d_2} = (1, 0, 1)$, $\vec{d_3} = (1, 1, 1)$. Compute all three cosines, rank, and explain why d_2 is penalised.

E2.3 **Length normalization.** Doc A has vector $(10, 0)$, doc B has $(1, 1)$, query $(1, 1)$. Euclidean distance ranks A closer to q than B under some centerings; cosine does not. Compute both and explain which better captures topical relevance.

E2.4 **Phrase via sets.** How would you support query `"cats dogs"` (adjacent) using positional postings? Sketch two postings lists with positions and describe the merge that checks adjacency.

E2.5 **Hybrid design.** Your system must support "must contain SC4021 and rank by similarity to *neural IR*." Design a two-stage pipeline (filter then rank) and state complexity per stage using postings sizes.

Chapter 3

Term Weighting: TF–IDF and Beyond

Not all words are equal: "the" appears everywhere and tells little; "PageRank" appears rarely and tells a lot. Weight words accordingly.

From zero: counts to weights step by step. We start from raw counts, see why they over-weight repeats and common words, then derive TF dampening, IDF rarity, and the TF–IDF product with length normalization.

Learning Outcomes

After this chapter you will be able to:

(L1) explain why raw term frequency is insufficient;

(L2) compute TF variants, IDF, and TF–IDF and interpret each factor;

(L3) apply length normalization and sublinear TF;

(L4) contrast TF–IDF with BM25 intuition (saturation + length penalty);

(L5) rank documents for a query using TF–IDF cosine.

3.1 Intuition: Frequency and Rarity

Intuition. Two signals determine a term's value for ranking. *Frequency*: if "retrieval" appears 10 times in d, d is more about retrieval than a doc where it appears once. But the 10th occurrence matters less than the 1st (diminishing returns). *Rarity*: "the" appears in every doc; matching it tells almost nothing. "PageRank" appears in few docs; matching it is highly discriminative. Good weighting rewards frequent *and* rare terms, penalises repeated common ones.

Raw count fails. Using $w(t,d) = count(t,d)$ gives undue advantage to long docs and to stopwords. A 10,000-word doc will outscore a 100-word focused doc even if the short doc is more relevant – because weights are not normalized. And "the" with count 200 will dominate "PageRank" with count 2.

3.2 Term Frequency Variants

Definition 3.1 (TF Variants)**.** Let $f_{t,d}$ be raw count of term t in doc d. Common TF dampenings:

$$\text{raw: } tf_{t,d} = f_{t,d} \tag{3.1}$$

$$\text{log: } tf_{t,d} = 1 + \log_{10}(f_{t,d}) \text{ if } f_{t,d} > 0,\ 0 \text{ otherwise} \tag{3.2}$$

$$\text{double-norm 0.5: } tf_{t,d} = 0.5 + 0.5 \cdot \frac{f_{t,d}}{\max_{t'} f_{t',d}} \tag{3.3}$$

$$\text{binary: } tf_{t,d} = 1 \text{ if } f_{t,d} > 0 \text{ else } 0. \tag{3.4}$$

Log TF is most common: $f = 1 \to 1$, $f = 10 \to 2$, $f = 100 \to 3$ – captures "first mention matters most." Double-norm additionally normalizes by doc's max frequency, bounding TF in $[0.5, 1]$.

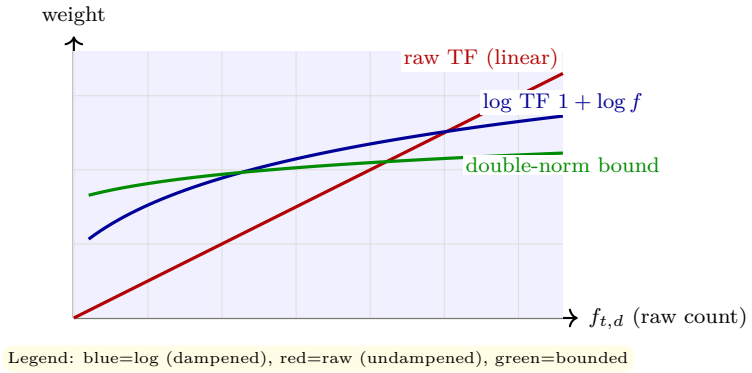


Figure 3.1: TF dampening: log and double-norm curb the influence of repeated terms.

3.3 Inverse Document Frequency

Definition 3.2 (IDF)**.** Let $N = |\mathcal{C}|$ and $df_t = |\{d : t \in d\}|$ (document frequency). Then

$$idf_t = \log_{10}\left(\frac{N}{df_t}\right). \tag{3.5}$$

If t appears in every doc, $df_t = N$, $idf = 0$ (no discrimination). If t appears in 1 of $N = 10^6$ docs, $idf = \log_{10} 10^6 = 6$ (very discriminative). Log dampens the ratio so rarity does not explode.

Definition 3.3 (TF–IDF Weight)**.**

$$w(t,d) = tf_{t,d} \times idf_t. \tag{3.6}$$

Document vector $\vec{d}$ has components $w(t,d)$; query vector $\vec{q}$ often uses $w(t,q) = (0.5 + 0.5 \cdot f_{t,q}/\max f_q) \cdot idf_t$ (double-norm for short queries). Ranking uses cosine over these weighted vectors.

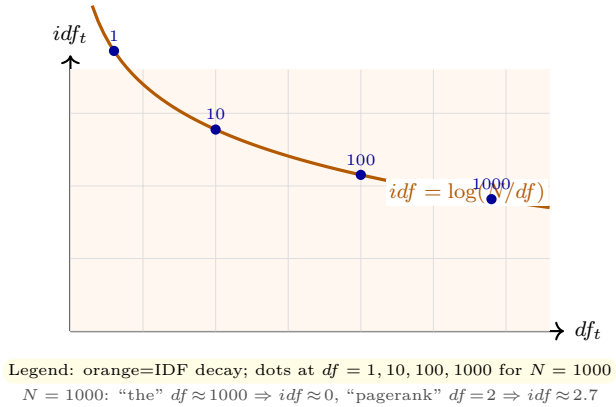


Figure 3.2: IDF falls as df rises: common terms approach zero weight.

3.4 Length Normalization

Long docs have higher $f_{t,d}$ simply because they have more words – not because they are more relevant. Cosine's denominator $\|\vec{d}\|$ already normalizes, but TF–IDF pipelines also normalize TF itself (double-norm) or use pivoted normalization: penalize long docs a bit more than cosine alone, because very long docs have lower term density and empirically benefit from extra penalty. We will revisit this in BM25 where length normalization is explicit via parameter b.

3.5 Worked Example: 3-Document TF–IDF

Collection $N = 3$: d_1='cats dogs', d_2='cats cats cats', d_3='dogs birds'. Vocabulary {cats, dogs, birds}. Query q='cats dogs'.

Use log TF and $\log_{10}$ IDF. For $N = 3$: cats $df = 2 \Rightarrow idf = \log \frac{3}{2} = 0.176$, dogs $df = 2 \Rightarrow 0.176$, birds $df = 1 \Rightarrow \log 3 = 0.477$. Thus **TF–IDF vectors**: d_1: cats 0.176, dogs 0.176; d_2: cats $(1 + \log 3) \times 0.176 = 0.26$, dogs 0; d_3: dogs 0.176, birds 0.477. Common terms have low IDF (blue/green), rare birds high (orange). Cosine ranking for q (cats+dogs): $\vec{q}$ has both terms ≈ 0.176 each. $\cos(q, d_1)$ is highest (both query terms present, length short), $\cos(q, d_3)$ next (one term but rare birds does not help), $\cos(q, d_2)$ lower despite high cats count because dampened TF and missing dogs.

Listing 3.1: TF–IDF from scratch and cosine ranking.

```
import math, numpy as np, collections
docs = ["cats dogs", "cats cats cats", "dogs birds"]
vocab = ["cats","dogs","birds"]
N = len(docs)
# document frequencies
df = {t: sum(1 for d in docs if t in d.split()) for t in vocab}
idf = {t: math.log10(N/df[t]) for t in vocab}
print("df:", df, "idf:", {k: round(v,3) for k,v in idf.items()})
def tfidf_vec(text):
    counts = collections.Counter(text.split())
    v = []
    for t in vocab:
        f = counts[t]
```

```
        tf = 1+math.log10(f) if f>0 else 0
        v.append(tf*idf[t])
    return np.array(v)
q = tfidf_vec("cats dogs")
for i,d in enumerate(docs):
    v = tfidf_vec(d)
    cos = q.dot(v) / (np.linalg.norm(q)*np.linalg.norm(v)) if np.linalg.norm(v)>0 else 0
    print(f"d{i+1} {v.round(3)} cos={cos:.3f}")
```

3.6 Beyond TF–IDF: BM25 Preview

TF–IDF's cosine treats TF linearly after log. BM25 (the production successor) adds *saturation*: $tf/(tf + k_1)$ flattens after $k_1 \approx 1.2$, and explicit length normalization with parameter $b \approx 0.75$: longer docs are penalised proportionally. BM25 score:

$$\text{BM25}(q, d) = \sum_{t \in q} idf_t \cdot \frac{f_{t,d}(k_1 + 1)}{f_{t,d} + k_1(1 - b + b \cdot |d|/\text{avgdl})}, \tag{3.7}$$

with $idf_t = \log \frac{N - df_t + 0.5}{df_t + 0.5}$. Figure below contrasts linear TF vs saturated.

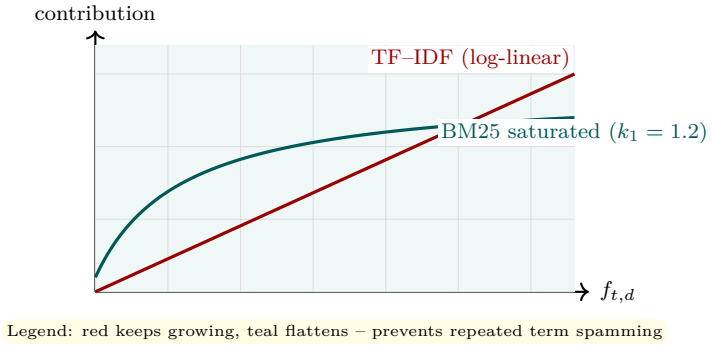


Figure 3.3: BM25 saturation vs. log TF: both dampen, but BM25 plateaus faster.

Listing 3.2: BM25 scoring for the same 3-doc collection.

```
import math
k1, b = 1.2, 0.75
N = 3
docs_tok = [d.split() for d in ["cats dogs", "cats cats cats", "dogs birds"]]
avgdl = sum(len(d) for d in docs_tok)/N
df = {"cats":2, "dogs":2, "birds":1}
def bm25_idf(t): return math.log((N - df[t] + 0.5)/(df[t] + 0.5) + 1)  # smoothed
def bm25_score(query, doc):
    score=0
    for t in query.split():
        f = doc.count(t)
        if f==0: continue
        idf = bm25_idf(t)
        num = f*(k1+1)
        den = f + k1*(1 - b + b*len(doc)/avgdl)
        score += idf * num/den
    return score
```

```
for i,d in enumerate(docs_tok):
    print(f"d{i+1} BM25 for 'cats dogs': {bm25_score('cats dogs', d):.3f}")
```

3.7 Choosing Heuristics: When to Use Which TF

Intuition. No single TF formula dominates everywhere. Short social media posts (few repeats) benefit less from log dampening than long news articles. Double-norm 0.5 shines when document lengths vary wildly; binary TF is surprisingly competitive for very short queries where repeating a term in the doc is rare anyway.

Rule of thumb. Start with log TF + $\log(N/df)$ IDF + cosine. If collection has huge length variance (e.g., web pages from 50 to 50,000 tokens), add pivoted normalization or switch to BM25 with $b \approx 0.75$. If queries are very short (1–2 terms), binary TF in docs plus IDF often suffices. Always cross-validate k_1, b for BM25 on a dev set (Ch. 5's MAP/NDCG) rather than guessing.

Remark 3.1 (IDF smoothing). Real systems avoid $\log(N/df)$ when df may be 0 (term not in collection) or $N \approx df$ (stopwords) by using $\log(1 + N/df)$ or probabilistic $idf = \log((N - df + 0.5)/(df + 0.5))$. The +0.5 avoids division by zero and negative idf for very frequent terms.

3.8 Chapter Notes

TF–IDF was formalised by Spärck Jones (1972, IDF) and Salton (SMART). It remains a strong baseline; neural models often complement rather than replace it (hybrid sparse-dense in Ch. 8). Always use the same tokenization for df and $f_{t,d}$; mismatch silently corrupts weights.

3.9 Exercises

E3.1 **TF variants.** Doc has counts {cats:5, dogs:1, birds:0}. Compute raw, log, double-norm 0.5, and binary TF for each term.

E3.2 **IDF.** $N = 10\,000$. Terms: "the" $df = 9\,800$, "retrieval" $df = 120$, "pagerank" $df = 15$. Compute idf ($\log_{10}$) and explain which term most discriminates.

E3.3 **TF–IDF by hand.** $N = 4$: d_1='cat dog', d_2='cat cat dog', d_3='dog bird', d_4='bird bird bird'. Use log TF, compute TF–IDF vectors and rank query "cat bird" by cosine.

E3.4 **Length normalization.** Two docs: A='cat' ×10, B='cat dog bird'. Query 'cat dog'. With raw TF, which doc wins? With log TF + cosine normalization, does the ranking flip? Compute both.

E3.5 **BM25 vs. TF–IDF.** For a term with $df = 5$, $N = 1000$, plot or compute BM25 contribution for $f = 1, 2, 5, 10, 20$ with $k_1 = 1.2$, $b = 0$, and compare to $tf \cdot idf$ with log TF. At what f does BM25 saturate noticeably?

Chapter 4

The Inverted Index: Making Retrieval Fast

Term-at-a-time in a hash table or document-at-a-time in postings? The inverted index is the engine that turns $O(N)$ scans into $O(p)$ lookups.

From zero: arrays to postings. We start from a scanned collection, build the dictionary + postings structure, handle large collections via blocking, support phrases via positions, and speed intersections with skips and compression.

Learning Outcomes

After this chapter you will be able to:

(L1) define dictionary, postings, docID, and positional index;

(L2) build an inverted index with a single-pass algorithm and analyse its cost;

(L3) handle phrase and proximity queries via positional merges;

(L4) apply gap encoding, variable-byte, and skip pointers;

(L5) reason about index compression ratios and query time trade-offs.

4.1 Why Not Scan? The Inverted Idea

Intuition. Naive retrieval scans every doc for each query: $O(N \cdot \text{avgdl})$ per query – impossible at $N = 10^9$. Instead, precompute: for each term t, store the sorted list of docs containing t – the *postings list*. Query `cats AND dogs` is then intersecting two sorted lists, touching only docs that contain at least one query term.

Structure. Dictionary: hash table or B-tree from term → pointer. Postings: per term, sorted docIDs plus payload (frequency, positions). At query time, fetch postings for query terms and merge.

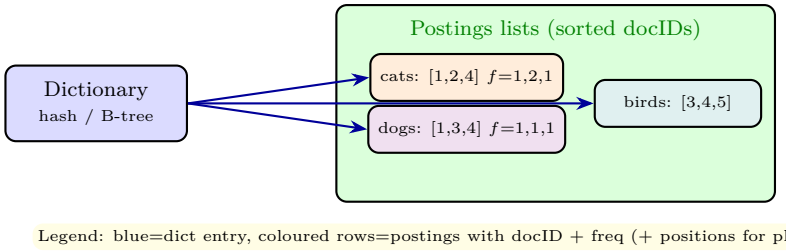


Figure 4.1: Inverted index: dictionary maps term to its postings list.

Definition 4.1 (Inverted Index)**.** For vocabulary $\mathcal{V}$, the index is $\{(t, P(t)) : t \in \mathcal{V}\}$ where

$$P(t) = [(d_1, f_{t,d_1}, [pos]),\ (d_2, f_{t,d_2}, [pos]),\ \ldots] \tag{4.1}$$

sorted by d_i. Positions $[pos]$ are stored only for phrase queries.

4.2 Building the Index

Single-pass (SPIMI). Stream docs one by one, tokenize, hash term → accumulate postings in memory. When memory fills, sort terms, write a *block* to disk. Finally merge blocks by term (k-way merge, like merge-sort). Complexity $O(T \log V)$ for T tokens; blocked variant (BSBI) does the same with external sorting.

Distributed. At web scale, partition by term (term-partitioned) or by doc (doc-partitioned, more common: each shard holds a full index for a doc subset; query fans out and merges top-k).

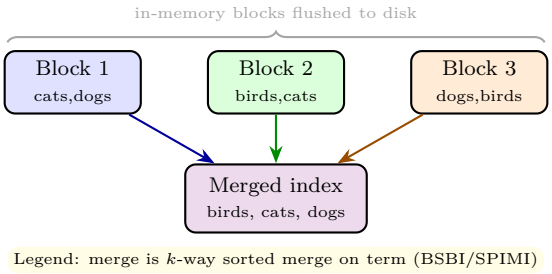


Figure 4.2: BSBI/SPIMI: build blocks in memory, then merge on disk.

Listing 4.1: SPIMI-style single-pass indexer for a tiny corpus.

```
import collections, re
corpus = {1:"cats dogs", 2:"cats cats", 3:"dogs birds", 4:"cats dogs birds"}
index = collections.defaultdict(list)  # term -> [(docID, freq, positions)]
for doc_id, text in corpus.items():
    toks = re.findall(r"\b\w+\b", text.lower())
    # positions for phrase support
    pos = collections.defaultdict(list)
    for i, t in enumerate(toks):
        pos[t].append(i)
    for t, plist in pos.items():
        index[t].append((doc_id, len(plist), plist))
# dictionary sorted
for t in sorted(index):
```

```
    print(t, index[t])
# cats [(1,1,[0]), (2,2,[0,1]), (4,1,[0])]
# dogs [(1,1,[1]), (3,1,[0]), (4,1,[1])]
```

4.3 Phrase and Proximity Queries

With only docIDs, `"cats dogs"` (adjacent) would falsely match a doc where cats and dogs are far apart. Positional postings fix it: check that positions differ by 1 (phrase) or within k (proximity NEAR/k).

Listing 4.2: Positional phrase query: are two terms adjacent?

```
def phrase_query(term1_postings, term2_postings):
    # postings: dict docID -> positions list
    result = []
    for doc in set(term1_postings) & set(term2_postings):
        p1, p2 = term1_postings[doc], term2_postings[doc]
        # two-pointer check for p2 == p1+1
        i=j=0
        p1s, p2s = sorted(p1), sorted(p2)
        while i < len(p1s) and j < len(p2s):
            if p2s[j] == p1s[i]+1:
                result.append(doc); break
            elif p2s[j] < p1s[i]+1: j+=1
            else: i+=1
    return result

cats_pos = {1:[0], 2:[0,1], 4:[0]}
dogs_pos = {1:[1], 3:[0], 4:[1]}
print(phrase_query(cats_pos, dogs_pos))  # [1,4] adjacent; doc 2 has cats cats no dogs
```

4.4 Compression and Skips

Postings are sorted docIDs: gaps are small. Store gaps $g_i = d_i - d_{i-1}$ (with $d_0 = 0$) instead of raw IDs. Gaps are encoded with variable-byte (VB) or gamma codes: small gaps use fewer bytes. Compression ratio $\approx$ 4:1 vs. 4-byte ints.

Skip pointers. For intersection of long lists, add skip links every $\sqrt{p}$ entries. If the other list's docID exceeds current block's max, jump forward – reducing comparisons from $O(p_1 + p_2)$ toward $O(\sqrt{p})$ skips.

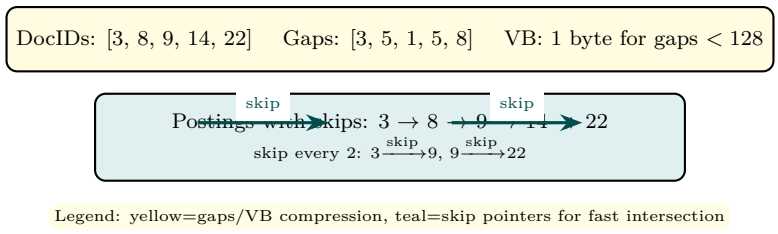


Figure 4.3: Gap encoding and skip pointers: space and time optimizations.

Production note. In Java ecosystems (Lucene/Elasticsearch) the same indexer is expressed with `Analyzer`, `IndexWriter`, and `Posting` objects: tokenize with `split("\W+")`, accumulate positions in a `Map<String,List<Integer>>`, then flush `Posting(docID,freq,pos)` into `Map<String,List<Posting>>` – structurally identical to the Python SPIMI above.

4.5 Complexity and Trade-offs

Indexing: $O(T)$ token processing + $O(V \log V)$ dictionary sort per block. Query: dictionary lookup $O(\log V)$ or $O(1)$ hash, plus merge of postings. For AND of m terms with postings sizes $p_1 \leq p_2 \leq \ldots$, naive merge is $O(\sum p_i)$; with skips and ordering shortest first, closer to $O(p_1 \log(p_m/p_1))$. Space: uncompressed $4N_{postings}$ bytes; compressed ≈ 1 byte per gap on average.

4.6 Updates, Deletion, and Dynamic Indexing

Intuition. Web collections change constantly: new pages appear, old ones disappear. Rebuilding the entire index from scratch nightly is feasible for moderate N but expensive at web scale. Dynamic structures append new docs to small in-memory indexes and merge them logarithmically (LSM-style).

Log-structured merges. Maintain C_0 (in-memory, mutable) and $C_1, C_2, \ldots$ (on-disk, immutable, exponentially larger). New docs go to C_0; when C_0 fills, merge it with C_1 if C_1 exists, cascading like binary addition. Query merges postings across all levels ($O(\log N)$ lists per term). Deletion marks doc as deleted in a bitset; postings are filtered at query time and physically purged during merges – identical to SC2207's LSM tree intuition.

Remark 4.1 (Fielded and faceted indexes). Real engines index fields separately (title, body, anchor text) with different weights, and facets (date, author) as separate dictionaries for filtering. The postings structure stays the same; only the dictionary is partitioned by field.

4.7 Fielded Search and Scoring at Query Time

Intuition. A doc is not flat text: title, body, anchor text, and metadata have different importance. A match in the title should count more than in the footer. Fielded scoring weights fields differently during ranking.

Scoring. For fields $f \in \{title, body, anchor\}$ with weights α_f, the score is $\sum_f \alpha_f \cdot score_f(q, d_f)$. At index time, fields can share postings (with field bits) or have separate indexes. Query processing becomes $O(\sum_f p_f)$ – still dominated by postings merges, but with $m \times fields$ lists. Modern engines use Block-Max indexes to skip docs whose partial field scores cannot enter top-k (WAND / MaxScore algorithms), achieving sub-millisecond query time even for long queries.

Remark 4.2 (DAAT vs. TAAT). Term-at-a-time (TAAT) scores one term's full postings before the next; document-at-a-time (DAAT) merges sorted docIDs and scores doc by doc. DAAT enables early termination

(once k good docs are found, lower-bound threshold prunes remaining). This choice is orthogonal to the earlier Boolean vs. vector distinction – both can use DAAT.

4.8 Chapter Notes

The inverted index is the most successful data structure in IR. Alternatives (suffix arrays, signature files) exist but rarely beat it for ranked retrieval. Modern engines (Lucene, Elasticsearch) store positions, frequencies, norms, and skip tables together in columnar formats. Dynamic indexing (add/delete docs) uses log-structured merges (LSM) – see SC2207 LSM trees.

4.9 Exercises

E4.1 **Build postings.** Corpus: d_1='cat dog cat', d_2='dog bird', d_3='cat bird bird'. Build dictionary + postings with frequencies and positions (docIDs sorted).

E4.2 **Positional phrase.** Using your index, answer `"cat dog"` and `"bird bird"` as phrase queries. Which docs match each? Show the position comparisons.

E4.3 **Skip intersection.** $P(a) = [1, 3, 5, 7, 9, 11, 13, 15]$, $P(b) = [2, 3, 6, 7, 10, 13, 14]$. Intersect with and without skips (skip every 3). Count comparisons each way.

E4.4 **VB encoding.** Gap sequence $[5, 1, 130, 2, 8]$. Encode with variable-byte (7 bits per byte, high bit = continuation). How many bytes total vs. 5×4 bytes uncompressed?

E4.5 **Distributed query.** Collection sharded doc-partitioned into 4 shards. Query 'cats dogs' fans out. Each shard returns top-3. How does the coordinator produce global top-10? What if shards use different IDFs – what correction is needed?

Chapter 5

Evaluation: Precision, Recall, MAP, and NDCG

If you cannot measure ranking quality, you cannot improve it. Precision and recall are the rulers; MAP and NDCG handle many queries and graded truth.

From zero: counting correct answers. We start from a single ranked list, define precision and recall, then average over recall levels and queries (MAP), and finally handle graded relevance with discounted gains (NDCG).

Learning Outcomes

After this chapter you will be able to:

(L1) compute precision, recall, F_1, and $P@k$ for a ranked list;

(L2) plot the precision–recall curve and define interpolated precision;

(L3) compute Average Precision (AP) and Mean Average Precision (MAP);

(L4) define DCG and NDCG for graded relevance and compute them;

(L5) describe pooling, Cranfield/TREC methodology, and significance.

5.1 Precision and Recall

Intuition. For query q, let Rel be the set of relevant docs (ground truth, size R) and Ret_k the top-k retrieved. Precision is "how clean is the answer?" Recall is "how complete is the answer?" A system that returns everything has recall 1 but terrible precision; one that returns one correct doc has precision 1 but low recall.

Definition 5.1 (Precision, Recall, F_1, $P@k$)**.**

$$\text{Precision } P = \frac{|Rel \cap Ret|}{|Ret|}, \qquad \text{Recall } R = \frac{|Rel \cap Ret|}{|Rel|}, \tag{5.1}$$

$$F_1 = \frac{2PR}{P+R} \text{ (harmonic mean)}, \qquad P@k \quad = \text{precision in top } k. \tag{5.2}$$

Example 5.1 (Single query)**.** $R = 4$ relevant docs $\{a, b, c, d\}$. System returns $[a, x, b, y, c]$ (top-5, x, y non-relevant). Then $|Rel \cap Ret_5| = 3$, $P = 3/5 = 0.6$, $R = 3/4 = 0.75$, $F_1 = 0.667$, $P@3 = 2/3 = 0.667$, $P@5 = 0.6$. As k increases, precision typically falls and recall rises or stays.

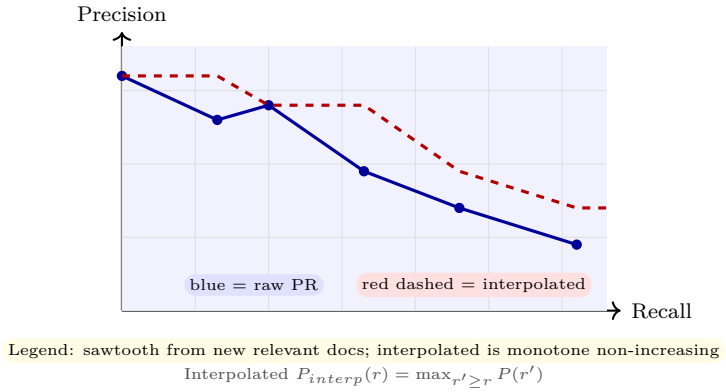


Figure 5.1: Precision–recall curve: raw (jagged) and 11-point interpolated (monotone).

Interpolated precision. IR curves are noisy. Define $P_{interp}(r) = \max_{r' \geq r} P(r')$ – the best precision achievable at or beyond recall r. This makes the curve monotone non-increasing and enables averaging across queries at fixed recall levels (e.g., 11-point: $0, 0.1, \ldots, 1.0$).

5.2 Average Precision and MAP

Intuition. A single $P@10$ ignores ordering within top-10 and recall beyond. AP rewards putting relevant docs early and accounts for all relevant docs.

Definition 5.2 (AP and MAP)**.** For query q with R_q relevant docs, let $rel(k) = 1$ if doc at rank k is relevant else 0. Then

$$AP(q) = \frac{1}{R_q} \sum_{k=1}^{n} P@k \times rel(k). \tag{5.3}$$

Only ranks where $rel(k) = 1$ contribute, weighted by precision there. For a set of queries Q,

$$MAP = \frac{1}{|Q|} \sum_{q \in Q} AP(q). \tag{5.4}$$

If a relevant doc is never retrieved, its contribution is 0.

Example 5.2 (AP by hand)**.** Relevant $\{a, b, c\}$ ($R = 3$). Ranking: $[a, x, b, y, c, z]$ (x, y, z non-rel). Then $P@1 = 1$, $P@3 = 2/3$, $P@5 = 3/5$. So $AP = (1 + 2/3 + 3/5)/3 = (1 + 0.667 + 0.6)/3 = 0.756$. If ranking were

$[x, a, y, b, z, c]$: $P@2 = 0.5$, $P@4 = 0.5$, $P@6 = 0.5$, $AP = (0.5 + 0.5 + 0.5)/3 = 0.5$ – lower because relevant docs appear later.

Listing 5.1: Precision, recall, and AP for a single query.

```
def precision_recall(retrieved, relevant, k=None):
    if k is not None: retrieved = retrieved[:k]
    tp = len(set(retrieved) & set(relevant))
    prec = tp / len(retrieved) if retrieved else 0
    rec = tp / len(relevant) if relevant else 0
    return prec, rec

def average_precision(retrieved, relevant):
    relevant = set(relevant)
    R = len(relevant)
    ap_sum, hits = 0.0, 0
    for k, doc in enumerate(retrieved, 1):
        if doc in relevant:
            hits += 1
            ap_sum += hits / k  # P@k at relevant rank
    return ap_sum / R if R else 0

rel = {"a","b","c"}
ret = ["a","x","b","y","c","z"]
print("P@3", precision_recall(ret, rel, k=3))
print("AP", round(average_precision(ret, rel),3))  # 0.756
ret2 = ["x","a","y","b","z","c"]
print("AP bad order", round(average_precision(ret2, rel),3))  # 0.5
```

5.3 Graded Relevance: DCG and NDCG

Binary relevant/non-relevant is coarse. Real judgments are graded (0=non, 1=marginal, 2=fair, 3=highly relevant). NDCG rewards putting highly relevant docs at the very top, with a discount for lower ranks.

Definition 5.3 (DCG / NDCG)**.** For graded relevance rel_i at rank i,

$$DCG@k = \sum_{i=1}^{k} \frac{2^{rel_i} - 1}{\log_2(i+1)} \quad \text{or simpler} \quad \sum_{i=1}^{k} \frac{rel_i}{\log_2(i+1)}. \tag{5.5}$$

The exponential numerator emphasises highly relevant. Ideal $IDCG@k$ is DCG@k for docs sorted by rel descending. Then

$$NDCG@k = \frac{DCG@k}{IDCG@k} \in [0, 1]. \tag{5.6}$$

Example 5.3 (NDCG)**.** Grades for top-5: $[3, 0, 2, 1, 0]$. $DCG@5 = (7/\log 2) + (0) + (3/\log 4) + (1/\log 5) + 0 = 7/1 + 3/2 + 1/2.32 = 9.93$ (using $2^{rel} - 1$). Ideal order $[3, 2, 1, 0, 0]$: $IDCG = 7/1 + 3/1.58 + 1/2 = 10.58$. $NDCG = 0.94$ – near ideal despite zeros because top doc is highly relevant.

Listing 5.2: DCG and NDCG with the exponential gain.

```
import math
```

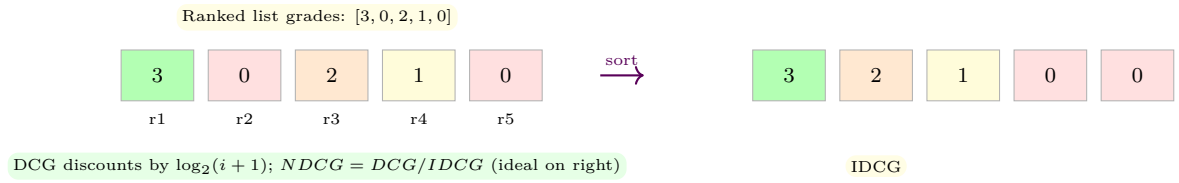


Figure 5.2: DCG: grades in ranked order vs. ideal order; discount penalises inversions.

```
def dcg(grades, k=None):
    if k is not None: grades = grades[:k]
    return sum((2**g - 1)/math.log2(i+2) for i,g in enumerate(grades))

grades = [3,0,2,1,0]
print("DCG@5", round(dcg(grades),3))
ideal = sorted(grades, reverse=True)
print("IDCG@5", round(dcg(ideal),3))
print("NDCG@5", round(dcg(grades)/dcg(ideal),3))  # 0.939
# Compare a worse ranking: highly relevant at rank 5
bad = [0,0,1,2,3]
print("NDCG bad", round(dcg(bad)/dcg(ideal),3))  # much lower
```

5.4 Cranfield, Pooling, and Significance

Real collections are too large to judge exhaustively. TREC/Cranfield methodology: assemble systems, pool their top-k (e.g., top-100 per query across systems), judge only pooled docs, treat unjudged as non-relevant. This biases evaluation against systems that did not contribute to the pool but works well when pools are deep and metrics like MAP/NDCG@10 focus on early ranks. Always report significance (paired t-test or randomization) over queries – a 0.02 MAP gain may be noise with 10 queries but real with 50.

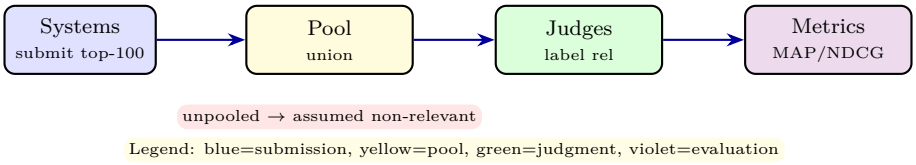


Figure 5.3: Pooling for large collections: judge only what systems retrieve near the top.

5.5 Micro vs. Macro and User-Oriented Metrics

Intuition. MAP is macro-averaged (average per query, then mean). Micro-averaged precision pools all query-doc pairs together, giving larger queries more weight. TREC uses macro – each information need counts equally, not each relevant doc. For imbalanced query sets (some queries have 5 relevant, others 500), the choice flips system rankings.

Other metrics. MRR (Mean Reciprocal Rank) cares only about the first relevant: $RR = 1/rank_{first}$. Good for known-item search ("find the homepage"). Success@k is binary: did we get at least one relevant in top-k? For user studies, time-to-first-relevant and effort (clicks) matter more than abstract precision.

Remark 5.1 (Statistical testing). With $|Q| = 50$ queries, a MAP difference of 0.03 may be noise. Use paired t-test, Wilcoxon, or randomization (Fisher) over per-query AP differences. Report $p < 0.05$ and effect size, not just raw MAP. Bootstrap confidence intervals over queries are increasingly standard.

5.6 Chapter Notes

MAP was the TREC workhorse for binary judgments; NDCG (Järvelin & Kekäläinen, 2002) is now standard for graded and top-heavy evaluation. Report P@10 and $NDCG$@10 for user-facing quality, MAP for recall-oriented tasks. Always average over queries, not over docs – queries are the experimental units. See Ch. 8 for how these metrics become training objectives.

5.7 Exercises

E5.1 P, R, F_1**.** $Rel = \{1, 2, 3, 4, 5\}$, $Ret = [1, 6, 2, 7, 3]$. Compute P, R, F_1, P@3, R@3. If you add doc 4 at rank 6, how do P and R change?

E5.2 **PR curve.** Relevant $\{a, b, c\}$. Ranking $[a, x, b, y, c]$. Compute $P@k$ and $R@k$ at each k, plot, and compute 11-point interpolated precision at $r = 0, 0.5, 1.0$.

E5.3 **AP/MAP.** Two queries: q_1 rel $\{a, b\}$, ret $[a, x, b]$; q_2 rel $\{c, d, e\}$, ret $[x, c, y, d, z, e]$. Compute $AP(q_1)$, $AP(q_2)$, MAP. Which query is harder?

E5.4 **NDCG.** Grades top-4: $[2, 0, 3, 1]$. Compute DCG@4 (exponential gain) and $NDCG$@4. What ranking would give $NDCG = 1$? Swap ranks 1 and 3, recompute $NDCG$ and quantify the drop.

E5.5 **Pooling bias.** A new system retrieves doc d_{new} at rank 2 that no pooled system found. Under "unjudged = non-relevant," what is its $NDCG$ effect? Propose two ways to mitigate pool bias when evaluating novel systems.

Chapter 6

Query Expansion and Relevance Feedback

Users write short, vague queries. Let the system ask "what did you mean?" by learning from judged documents and language itself.

From zero: closing the vocabulary gap. We start from mismatch (synonymy/polysemy), let users or pseudo-judgments label results, then move the query vector toward relevance (Rocchio) and expand with related terms.

Learning Outcomes

After this chapter you will be able to:

(L1) diagnose vocabulary mismatch and query underspecification;

(L2) apply Rocchio feedback with explicit and pseudo-relevance judgments;

(L3) expand queries via thesaurus and embedding neighbours;

(L4) evaluate expansion while guarding against query drift;

(L5) weigh interactive vs. automatic feedback trade-offs.

6.1 Why Queries Fail

Intuition. The user types `car`. The relevant doc uses `automobile`. No term overlaps, so TF–IDF scores 0 – a false negative (synonymy). Conversely, `bank` could mean river or finance – a false positive (polysemy). Studies show 40–60% of relevant docs share no exact query term. Expansion bridges this gap by adding related terms; feedback learns which terms actually correlate with relevance. Conceptually, *synonymy causes misses* (query `car` misses doc with `automobile`) while *polysemy causes false hits* (query `bank` hits both river and finance docs incorrectly).

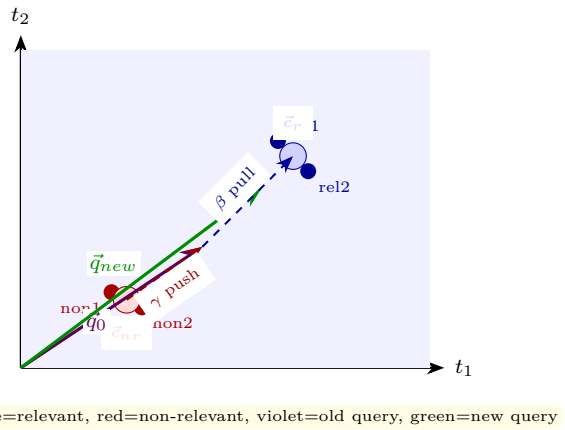
6.2 Rocchio Feedback

Intuition. Show the user top results, let them mark relevant vs. non-relevant, then move the query vector toward the relevant centroid and away from the non-relevant centroid. Like adjusting aim after seeing where shots landed.

Formal. Let D_r be relevant docs, D_{nr} non-relevant, centroids $\vec{c}_r$, $\vec{c}_{nr}$. Rocchio update:

$$\vec{q}_{new} = \alpha\,\vec{q}_0 + \beta\,\frac{1}{|D_r|}\sum_{d\in D_r}\vec{d} - \gamma\,\frac{1}{|D_{nr}|}\sum_{d\in D_{nr}}\vec{d}, \tag{6.1}$$

with typical $\alpha = 1$, $\beta = 0.75$, $\gamma = 0.15$. Weights clipped at 0 (negative weights zeroed). Intuition: β pulls toward what worked, γ pushes from what failed, α remembers the original ask.



Legend: blue=relevant, red=non-relevant, violet=old query, green=new query

Figure 6.1: Rocchio: query moves toward relevant centroid, away from non-relevant centroid.

Example 6.1 (Rocchio numeric)**.** Vocab {cat, dog}. $\vec{q}_0 = (1, 0)$, rel doc $(1, 1)$, non-rel doc $(0, 1)$, $\alpha = 1, \beta = 0.75, \gamma = 0.25$. Then $\vec{q}_{new} = 1 \cdot (1, 0) + 0.75 \cdot (1, 1) - 0.25 \cdot (0, 1) = (1.75, 0.5)$. Weight of "dog" rose from 0 to 0.5 because relevant docs contained it.

Listing 6.1: Rocchio update and re-ranking.

```
import numpy as np
q0 = np.array([1., 0.])         # query "cat"
rel = [np.array([1.,1.]), np.array([1.,0.5])]  # relevant docs
non = [np.array([0.,1.])]                       # non-relevant
alpha, beta, gamma = 1.0, 0.75, 0.25
cr = sum(rel)/len(rel)
cnr = sum(non)/len(non)
q_new = alpha*q0 + beta*cr - gamma*cnr
q_new = np.maximum(q_new, 0)  # clip negatives
print("q_new:", q_new.round(3))
def cosine(a,b): return a.dot(b)/(np.linalg.norm(a)*np.linalg.norm(b))
for i,d in enumerate(rel+non):
    print(f"doc{i} cos old={cosine(q0,d):.3f} new={cosine(q_new,d):.3f}")
```

6.3 Pseudo-Relevance and Expansion

Explicit feedback requires user effort. *Pseudo-relevance feedback* (blind) assumes top-k retrieved are relevant and feeds them to Rocchio automatically – often helps, sometimes hurts (if top-k is poor, drift amplifies noise). Better: extract expansion terms by scoring terms in D_r vs. collection (e.g., by $tf \cdot idf$ or $P(t|D_r)/P(t|\mathcal{C})$), add top-$m$ to query with lower weight.

Thesaurus and embeddings. A thesaurus (WordNet) adds synonyms/hyponyms; distributional thesaurus or word embeddings add nearest neighbours in vector space (`car` → automobile, vehicle). Modern query expansion uses embedding similarity or LLM-generated paraphrases, then re-ranks.

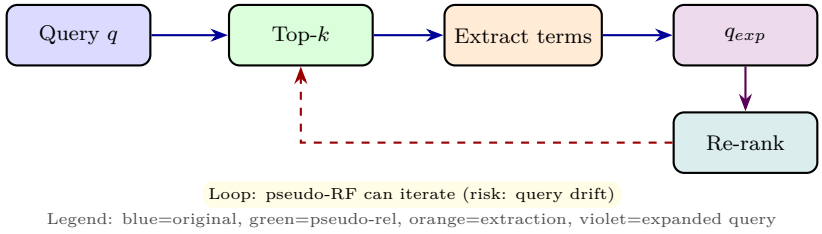


Figure 6.2: Pseudo-relevance feedback loop: assume top-k relevant, expand, re-retrieve.

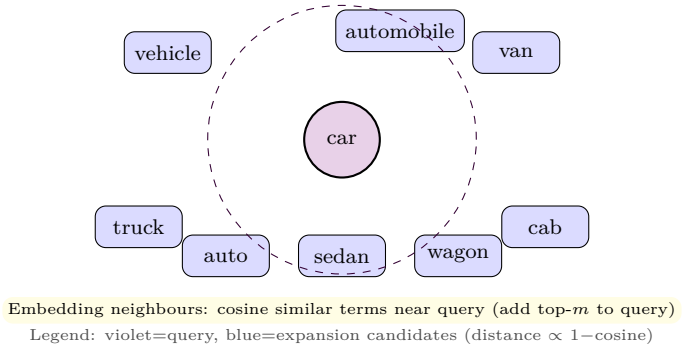


Figure 6.3: Embedding-based expansion: nearest neighbours of a query term in embedding space.

Listing 6.2: Embedding-style expansion by co-occurrence similarity.

```
import collections, math
docs = ["car automobile vehicle", "car van truck", "river bank finance bank", "automobile
    sedan wagon"]
vocab = ["car","automobile","vehicle","van","truck","bank","river","finance"]
# toy co-occurrence: Jaccard of doc sets
from collections import defaultdict
doc_sets = {t: {i for i,d in enumerate(docs) if t in d} for t in vocab}
def jaccard(a,b):
    s = doc_sets[a] & doc_sets[b]
    u = doc_sets[a] | doc_sets[b]
    return len(s)/len(u) if u else 0
query = "car"
scores = sorted([(t, jaccard(query,t)) for t in vocab if t!=query], key=lambda x: -x[1])
print(scores[:3])  # [('automobile', ...), ('vehicle', ...)]
expanded = query + " " + " ".join(t for t,_ in scores[:2])
print("expanded query:", expanded)
```

Danger: query drift. Adding `bank:finance` terms to a river-bank query hurts. Mitigations: weight expansion terms lower than originals, require terms to co-occur with multiple query terms, or use relevance model $P(t|R)$ rather than raw similarity.

6.4 Relevance Models and RM3

Intuition. Rocchio is vector-space; the probabilistic counterpart is the *relevance model* $P(w|R)$: what is the distribution of words among relevant docs? Estimate $P(w|R) \approx \sum_{d \in D_r} P(w|d)P(d|R)$ (often $P(d|R)$ uniform over pseudo-relevant set), then interpolate the original query model $P(w|Q)$ with it: $P(w|Q') = (1-\lambda)P(w|Q) + \lambda P(w|R)$. RM3 does this with Dirichlet smoothing.

Formal. Let $P_{ML}(w|d) = f_{w,d}/|d|$. Then $\hat{P}(w|R) \propto \sum_{d \in D_r} P_{ML}(w|d)\, P(q|d)$ where $P(q|d) = \prod_{w \in q} P_{ML}(w|d)$ weights docs by query likelihood. Top words under $\hat{P}(w|R)$ that were not in q become expansion terms, with weight $\hat{P}(w|R)$. In practice RM3 gives 5–12% MAP gains on TREC and is the default pseudo-RF in Anserini/Lucene toolkits.

Remark 6.1 (Interactive vs. automatic). Interactive feedback (user marks relevance) is gold: 30–50% gains with one iteration. Pseudo-RF is free but risky. Modern systems blend both: show top results with "did you mean / related searches" that are essentially precomputed expansions the user can accept or ignore.

6.5 Evaluation of Expansion and Query Performance Prediction

Intuition. Expansion is not always good. Short, ambiguous queries ("jaguar") gain more than long specific ones ("Cranfield retrieval evaluation 1967"). Predicting when to expand – query performance prediction (QPP) – avoids hurting already-good queries.

QPP signals. Pre-retrieval: query length, average *idf*, SCQ (similarity of collection to query). Post-retrieval: clarity score (KL between query language model and collection), *NQC* (normalized query commitment: std of top scores). If top scores are tightly clustered (low NQC), expansion may help; if one doc dominates, the query is already clear. Production systems gate expansion on NQC or on initial *NDCG* estimate.

Remark 6.2 (Personalized expansion). Two users typing "java" have different intents (coffee vs. language) based on history. Personalized expansion biases $P(w|R)$ toward the user's past relevance judgments – essentially Rocchio where D_r is the user's click history. This connects Ch. 6's feedback to Ch. 8's learned personalization.

6.6 Chapter Notes

Rocchio (1971, SMART) remains the clearest formulation of feedback; probabilistic relevance feedback (Robertson/Sparck Jones) gives a complementary derivation. Modern pseudo-RF (RM3) and neural query expansion (ANCE, Query2Doc) revive the idea with learned similarities. Always evaluate expansion on a held-out query set: gains of 5–15% MAP are typical when done well, losses of similar magnitude when drift is unmanaged.

6.7 Exercises

E6.1 **Rocchio by hand.** $\vec{q}_0 = (1, 1, 0)$, rel docs $(1, 1, 0), (1, 0, 1)$, non-rel $(0, 1, 1)$, $\alpha = 1, \beta = 1, \gamma = 0.5$. Compute $\vec{q}_{new}$ and re-rank the three docs by cosine before vs. after.

E6.2 **Pseudo-RF risk.** Top-3 for "jaguar" are [car page, animal page, car page]. Pseudo-RF assumes all three relevant. Which expansion terms would be added and which sense would drift? How would you detect and limit drift?

E6.3 **Expansion scoring.** Relevant set word counts: "retrieval:5, neural:4, the:20, search:6," collection counts "retrieval:50, neural:40, the:10000, search:200." Score terms by $P(t|R)/P(t|C)$ and pick top-2 to add.

E6.4 **Embedding neighbours.** Using any pretrained embedding (or Jaccard above), list 5 neighbours of "bank" and partition them by sense (finance vs. river). How would you expand query "bank deposit" vs. "river bank" differently?

E6.5 **Weight vs. add.** Argue why expansion terms should generally have lower weight than original query terms. Give a counterexample where an expansion term deserves equal weight.

Chapter 7

Link Analysis: PageRank and HITS

Content tells what a page says; links tell what others think of it. Authority flows along edges.

From zero: graphs to authority. We start from a web graph (nodes=pages, edges=links), formalise a random surfer, derive PageRank as its stationary distribution, and contrast with query-dependent hubs and authorities (HITS).

Learning Outcomes

After this chapter you will be able to:

(L1) model the web as a directed graph and handle dangling nodes;

(L2) derive PageRank with teleportation and compute it via power iteration;

(L3) explain the effect of damping factor α;

(L4) contrast PageRank (global) vs. HITS (hubs/authorities, per query);

(L5) discuss spam, efficiency, and personalization.

7.1 The Web as a Graph

Intuition. A page that many important pages link to is likely important – recursively. A link is an endorsement, but endorsements from important endorsers count more. This recursive definition is exactly an eigenvector problem.

Formal. Let $G = (V, E)$ with $|V| = N$. Define transition matrix M where $M_{ji} = 1/outdeg(i)$ if $i \to j$, else 0 (column-stochastic: each column sums to 1, except dangling nodes with $outdeg = 0$). A random surfer at node i follows a random out-link uniformly. Without teleportation, the surfer's distribution $\vec{p}_t$ evolves as $\vec{p}_{t+1} = M\vec{p}_t$. The stationary distribution (if it exists) would satisfy $\vec{p} = M\vec{p}$.

Problem: dangling nodes leak probability (zero columns), and the graph may be disconnected or periodic (rank sinks trap the surfer).

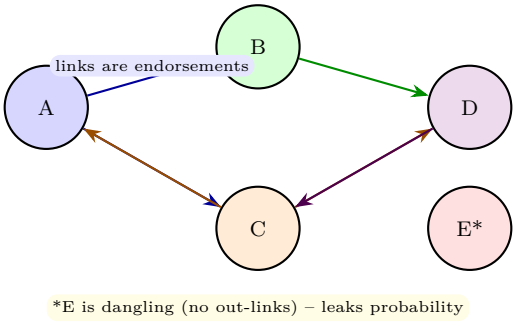


Figure 7.1: Web graph with 5 nodes; E is dangling. Node sizes will later encode PageRank.

7.2 PageRank: Random Surfer with Teleportation

Fix the leak and disconnectedness by teleportation: with probability α follow a link, with probability $1-\alpha$ jump to a random node (uniform). Standard $\alpha = 0.85$.

Definition 7.1 (PageRank)**.** Let $\vec{v} = \frac{1}{N}\vec{1}$ (teleport vector). Then PageRank $\vec{pr}$ satisfies

$$\vec{pr} = \alpha M \vec{pr} + (1-\alpha)\vec{v}, \qquad \sum_i pr_i = 1,\ pr_i \geq 0. \tag{7.1}$$

Equivalently $\vec{pr}$ is the stationary distribution of $M' = \alpha M + (1-\alpha)\vec{v}\vec{1}^T$. Solved by power iteration: $\vec{pr}^{(t+1)} = \alpha M \vec{pr}^{(t)} + (1-\alpha)\vec{v}$, starting from uniform, converging geometrically at rate α.

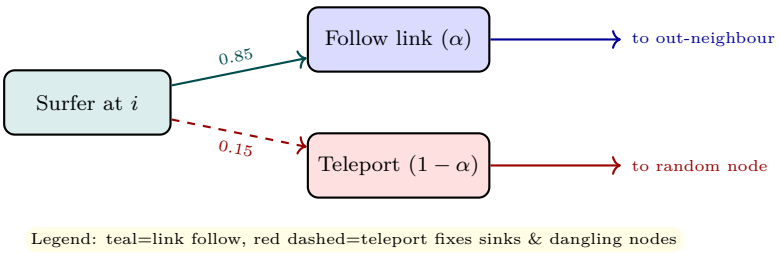


Figure 7.2: Random surfer: with prob. α follow a link, otherwise teleport.

Example 7.1 (3-node PageRank by hand)**.** Graph: $A \to B$, $B \to C$, $C \to A, B$ (so outdeg $A1, B1, C2$). $M = \begin{pmatrix} 0 & 0 & 1/2 \\ 1 & 0 & 1/2 \\ 0 & 1 & 0 \end{pmatrix}$ columns A, B, C. Let $\alpha = 0.85$, $\vec{v} = (1/3, 1/3, 1/3)^T$. Start uniform $(0.333, 0.333, 0.333)$. One iteration: $\vec{pr}^{(1)} = 0.85 M \vec{pr}^{(0)} + 0.15\vec{v}$. Compute $M\vec{pr}^{(0)} = (0.167, 0.5, 0.333)^T$, so $\vec{pr}^{(1)} = (0.192, 0.475, 0.333)^T$. Iterate until convergence; stationary $\approx (0.29, 0.39, 0.32)$ – B highest because two nodes point to it (directly or via C's split).

Listing 7.1: Power iteration for PageRank with dangling handling.

```
import numpy as np
def pagerank(adj, alpha=0.85, max_iter=100, tol=1e-6):
```

```
    # adj: dict node -> list of out-neighbours; nodes 0..N-1
    N = len(adj)
    # build column-stochastic M, handle dangling: uniform column
    M = np.zeros((N,N))
    for j, outs in adj.items():
        if not outs:  # dangling
            M[:, j] = 1/N
        else:
            for i in outs: M[i, j] = 1/len(outs)
    v = np.ones(N)/N
    pr = np.ones(N)/N
    for it in range(max_iter):
        nxt = alpha * M.dot(pr) + (1-alpha)*v
        if np.linalg.norm(nxt-pr, 1) < tol:
            break
        pr = nxt
    return pr

adj = {0:[1], 1:[2], 2:[0,1]}  # A->B, B->C, C->A,B
print(pagerank(adj).round(3))  # ~[0.29 0.39 0.32]
# effect of damping
for a in [0.5, 0.85, 0.95]:
    print(f"alpha={a}", pagerank(adj, alpha=a).round(3))
```

Effect of α. α near 1 respects graph structure strongly but converges slowly (α^t) and is sink-sensitive; α near 0 approaches uniform and loses link information. 0.85 is a practical sweet spot. For dangling nodes, the uniform column above is equivalent to teleporting when stuck – otherwise probability mass would vanish.

7.3 HITS: Hubs and Authorities

PageRank is query-independent (global). HITS (Kleinberg) is query-dependent: given a query, build a focused subgraph (e.g., top-k text matches plus their neighbours), then find two scores per node.

Definition 7.2 (HITS)**.** Each node i has hub score h_i and authority score a_i, iterated:

$$a_i \leftarrow \sum_{j:j\to i} h_j, \qquad h_i \leftarrow \sum_{j:i\to j} a_j, \tag{7.2}$$

then normalize $\|a\|_2 = \|h\|_2 = 1$. Authorities are pointed to by good hubs; hubs point to good authorities – mutual reinforcement. Converges to principal eigenvectors of A^TA and AA^T.

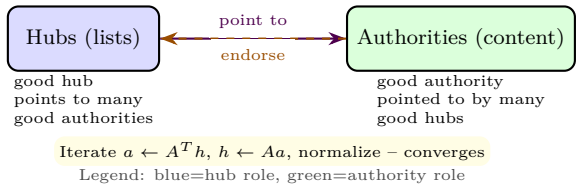


Figure 7.3: HITS mutual reinforcement: hubs and authorities improve each other.

Listing 7.2: HITS iteration on a tiny graph.

```python
import numpy as np
# adjacency matrix A[i,j]=1 if j->i (column j points to row i)
A = np.array([[0,0,1],[1,0,1],[0,1,0]], dtype=float)  # same graph as before
N = A.shape[0]
h = np.ones(N)/np.linalg.norm(np.ones(N))
a = np.ones(N)/np.linalg.norm(np.ones(N))
for _ in range(20):
    a = A.T.dot(h) if False else A.dot(h)  # careful: A is defined column->row
    # Actually A as defined: A[i,j]=1 if j->i, so a = A @ h is correct
    a = A.dot(h); a /= np.linalg.norm(a) + 1e-12
    h = A.T.dot(a); h /= np.linalg.norm(h) + 1e-12
print("authority:", a.round(3))
print("hub:      ", h.round(3))
```

PageRank vs. HITS: PageRank is global, precomputed and robust; HITS is per-query and captures bipartite hub/authority structure but is prone to topic drift (a good hub on a slightly different topic can hijack authority). Modern IR uses PageRank-like static scores as features inside learning-to-rank (Ch. 8) rather than standalone rankers.

7.4 Spam, Scale, Personalization

Link spam (link farms, bought links) inflates PageRank. Defenses: TrustRank (teleport biased to trusted seeds), spam mass estimation, and content+link joint models. At scale, power iteration is $O(|E|)$ per iteration – feasible for 10^{10} edges with distributed sparse multiplication (e.g., MapReduce). Personalized PageRank replaces uniform $\vec{v}$ with a user-specific distribution (e.g., teleport to user's bookmarks) – basis for recommender systems.

7.5 Topic-Sensitive and Personalized PageRank

Intuition. Uniform teleport assumes every page is equally good to jump to. But a user interested in "IR research" should teleport preferentially to IR pages, not gardening pages. Topic-sensitive PageRank replaces $\vec{v}$ with a biased distribution.

Formal. Let topics $T_1, \ldots, T_k$ with biased teleport vectors $\vec{v}_{T_i}$ (uniform over pages in topic T_i). Precompute k topic-specific PageRanks $\vec{pr}_{T_i}$. At query time, classify q into topics with weights $\beta_i(q)$ and *compose*: $\vec{pr}(q) = \sum_i \beta_i(q)\, \vec{pr}_{T_i}$ – answering in milliseconds without per-query power iteration. Personalized PageRank is the extreme where $\vec{v}$ is concentrated on a user's own pages or bookmarks; computation uses Monte Carlo random walks rather than full iteration.

Remark 7.1 (Beyond links: social and bipartite graphs). The same power iteration ranks products by purchase graphs, authors by citation graphs, and users by follower graphs. HITS-style hub/authority appears in bipartite query-click graphs (queries as hubs, clicked docs as authorities) – the basis for click-based re-ranking before neural models.

7.6 Convergence, Implementation, and Matrix View

Intuition. Naive iteration is $O(|E|)$ per step: each edge contributes one fraction of its source's score. For $N = 10^9$, $|E| \approx 10^{10}$, one iteration is already heavy, but convergence to 10^{-6} L_1 takes $\approx \log(10^{-6})/\log(\alpha) \approx 85$ iterations at $\alpha = 0.85$ – feasible in distributed Spark/GraphX but not per query.

Matrix and eigenvector view. Let $P = M'$ be the Google matrix. Then $\vec{pr}$ is the principal eigenvector ($P\vec{pr} = \vec{pr}$) with eigenvalue 1; all other eigenvalues have $|\lambda| \leq \alpha < 1$ (Perron–Frobenius), so power iteration contracts errors by α each step. Second eigenvalue bound explains the α^t convergence rate and why $\alpha = 0.85$ is a tradeoff: larger α keeps graph structure but needs more iterations.

Remark 7.2 (Sparse representation). Never form dense M. Represent adjacency as compressed sparse column (CSC) and compute $M\vec{pr}$ as scatter-gather: each node i sends $pr_i/outdeg(i)$ to its neighbours. Dangling contributions are summed once as uniform teleport mass – an $O(N)$ add, not $O(N^2)$.

7.7 Chapter Notes

PageRank (Page & Brin, 1998) and HITS (Kleinberg, 1999) founded link analysis; both are eigenvector centralities (cf. SC2001 spectral ideas). PageRank's teleport is mathematically PageRank's "damping" – identical to the random surfer and also to regularized Markov chains (SC2000). For implementation, never materialise dense M; use sparse adjacency and handle dangling columns implicitly.

7.8 Exercises

E7.1 **PageRank 3 nodes.** Graph $A \to B \to C \to A$. With $\alpha = 1$ (no teleport), what is PageRank? With $\alpha = 0.85$, do 2 power iterations from uniform and compare.

E7.2 **Dangling node.** Graph $A \to B$, B dangling. Write M with the dangling fix (uniform column for B) and compute one PageRank iteration from uniform with $\alpha = 0.85$.

E7.3 **Damping effect.** For graph in Fig. 7.1, qualitatively how does PageRank of E (dangling) change as $\alpha \to 1$ vs. $\alpha \to 0$? Explain via the teleport term.

E7.4 **HITS one step.** Focused graph: hub $h_1 \to a_1, a_2$, hub $h_2 \to a_1$, authority a_1 pointed by both hubs. Start $h = (1, 1)/\sqrt{2}$, $a = (0, 0)$? Instead start uniform $h = a = (1, 1, 1)/\sqrt{3}$ for 3 nodes and do one HITS update; which node becomes top authority?

E7.5 **Spam resistance.** A spammer adds 100 dummy pages all linking to victim V. With uniform teleport, how does $PR(V)$ change? Propose a teleport-bias defense and explain why it limits spam gain.

Chapter 8

Learning to Rank and Neural IR

From hand-tuned weights to learned rankers: let data decide how to combine signals, and let embeddings bridge the vocabulary gap neurally.

From zero: ranking as learning. We start from feature vectors per query-doc pair, frame ranking as supervised learning with pointwise/pairwise/listwise losses, then replace sparse terms with dense embeddings and bi-/cross-encoders, and fuse both worlds.

Learning Outcomes

After this chapter you will be able to:

(L1) describe pointwise, pairwise, and listwise learning-to-rank and their losses;

(L2) design IR features (BM25, PageRank, proximity, click signals) for a ranker;

(L3) contrast bi-encoders, cross-encoders, and late interaction (ColBERT);

(L4) explain dense retrieval, ANN search (HNSW/IVF), and hybrid sparse-dense fusion;

(L5) evaluate and diagnose neural rankers with NDCG/MAP and bias considerations.

8.1 Ranking as Supervised Learning

Intuition. Instead of hand-tuning score $= w_1 \cdot \text{BM25} + w_2 \cdot \text{PageRank}$, learn w (or a nonlinear ranker) from labeled query-doc pairs. Training data: (q, d, rel) with graded relevance. At test time, score all candidates for a new q and sort.

Three framings.

Pointwise Predict rel directly: regression/classification loss $\ell(f(q,d), rel)$. Ignores ordering among docs for

the same query.

Pairwise Predict preference: for $rel_i > rel_j$, want $f(q, d_i) > f(q, d_j)$. Loss e.g. RankNet: $\log(1 + \exp(-(s_i - s_j)))$. Directly optimises pairwise inversions.

Listwise Optimise the whole ranked list's metric (e.g., NDCG). LambdaMART does this by weighting pairwise gradients λ_{ij} by NDCG gain from swapping i and j – the most successful traditional LTR method.

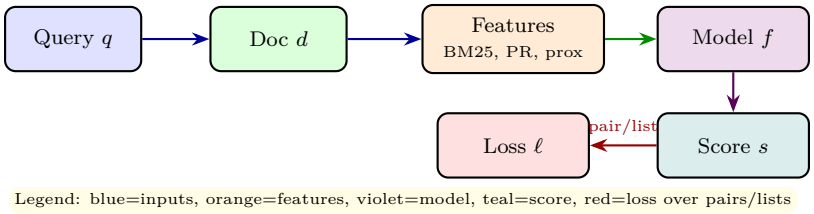


Figure 8.1: Learning-to-rank pipeline: per-pair features → scoring model → metric-aware loss.

Definition 8.1 (Features for LTR)**.** Typical features per (q, d): BM25(q, d) (Ch. 3), TF–IDF cosine (Ch. 2), document length, PageRank (Ch. 7), URL depth, proximity (min window covering query terms), click-through rate, and fielded scores (title vs. body). Feature vector $\phi(q, d) \in \mathbb{R}^p$, score $s = f(\phi)$ with f linear, tree ensemble (LambdaMART), or neural.

Listing 8.1: Toy pairwise ranker: learn linear weights with RankNet-style loss.

```
import numpy as np
# features: [BM25, PageRank, proximity]
# query "cats dogs": 3 docs with graded rel 2,1,0
X = np.array([[2.1, 0.3, 0.9], [1.2, 0.8, 0.2], [0.4, 0.1, 0.1]])
y = np.array([2, 1, 0])
w = np.zeros(3)
lr = 0.1
for epoch in range(60):
    grad = np.zeros(3)
    for i in range(len(y)):
        for j in range(len(y)):
            if y[i] > y[j]:
                si, sj = X[i].dot(w), X[j].dot(w)
                # RankNet gradient for this pair
                p = 1/(1+np.exp(-(si - sj)))  # prob i > j
                g = (1 - p)  # want p->1
                grad += g * (X[i] - X[j])
    w += lr * grad / 10
print("learned w:", w.round(3))
scores = X.dot(w)
print("scores:", scores.round(3), "order:", np.argsort(-scores))
# Should rank doc0 > doc1 > doc2 matching y
```

8.2 Neural IR: Dense Retrieval

Sparse models (BM25/TF–IDF) need exact term overlap. Dense retrieval embeds query and doc into a continuous space where `car` and `automobile` are near each other, then ranks by cosine/dot product of embeddings.

Trained so that relevant pairs are close, non-relevant far (contrastive loss).

Bi-encoder vs. cross-encoder.

Bi-encoder Encode q and d independently: $\vec{q} = E_q(q)$, $\vec{d} = E_d(d)$, score $= \vec{q} \cdot \vec{d}$ (or cosine). Fast: all doc vectors precomputed and ANN-indexed.

Cross-encoder Concatenate $[q; d]$ and score jointly with a transformer: $s = Transformer([q; d])$. Accurate (full attention between query and doc terms) but slow – must run per pair at query time.

ColBERT (late interaction) Token-level embeddings for both, then sum of max-sim per query token: $s = \sum_i \max_j \vec{q}_i \cdot \vec{d}_j$. Middle ground: decomposable yet expressive.

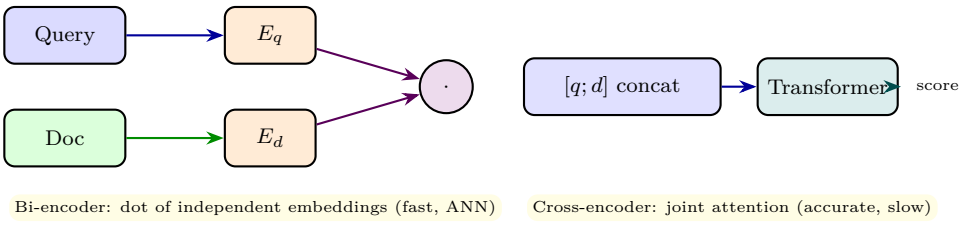


Figure 8.2: Bi-encoder (dual encoders + dot) vs. cross-encoder (joint scoring).

Listing 8.2: Bi-encoder toy with TF–IDF vectors as "embeddings" and ANN via brute force.

```
import numpy as np
# Toy: use 4-D ‘‘embeddings’’ (in practice use transformer vectors)
docs_emb = np.array([[0.9,0.1,0.2,0.0],[0.2,0.8,0.1,0.3],[0.1,0.1,0.9,0.8]])
query_emb = np.array([0.85,0.15,0.2,0.05])  # near doc0
# cosine ranking
def cos(a,b): return a.dot(b)/(np.linalg.norm(a)*np.linalg.norm(b))
scores = [cos(query_emb, d) for d in docs_emb]
print([round(s,3) for s in scores])  # [0.98, 0.35, 0.21]
# ANN sketch: IVF/HNSW would index docs_emb for sublinear search at scale
# Hybrid fusion: weighted sum of BM25 score + dense score
bm25 = np.array([2.1, 1.0, 0.3])
dense = np.array(scores)
hybrid = 0.6*bm25/np.max(bm25) + 0.4*dense  # normalize then fuse
print("hybrid:", hybrid.round(3))
```

8.3 ANN and Hybrid Retrieval

Dense vectors live in $\mathbb{R}^{768}$ (BERT-size). Brute-force scanning $N = 10^9$ vectors per query is too slow. Approximate Nearest Neighbour (ANN) indexes – IVF (inverted file over clusters), HNSW (hierarchical navigable small world graphs) – give sub-millisecond top-k with $> 95\%$ recall. Production systems are *hybrid*: retrieve top-k sparse (BM25) and top-k dense (ANN), then fuse (weighted sum or learned ranker) and optionally re-rank top-50 with a cross-encoder – the classic retrieve-and-rerank pipeline.

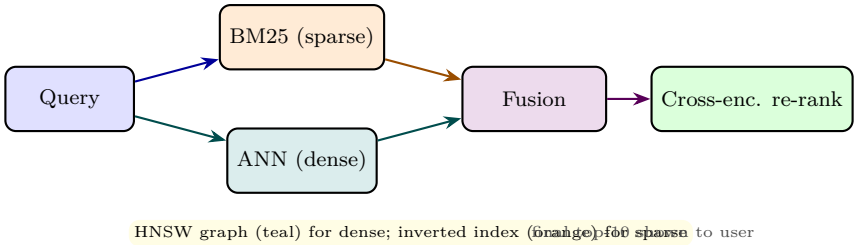


Figure 8.3: Hybrid retrieve-and-rerank: sparse + dense retrieve, fuse, then cross-encoder re-ranks the shortlist.

8.4 Evaluation, Data, and Pitfalls

Train neural rankers on MS MARCO, TREC DL, or click logs; evaluate with NDCG/MAP (Ch. 5). Pitfalls: data leakage (eval queries in training), position bias in clicks, efficiency vs. quality (cross-encoders are 100× slower than bi-encoders), and corpus drift (embeddings stale as collection grows – need re-indexing). Always report both quality and latency (queries per second, index size).

8.5 Training Data, Negatives, and Contrastive Learning

Intuition. Neural rankers need negatives: for query "neural IR," doc "neural networks for IR" is positive, but which negatives teach the model? Random negatives are trivial (easy to distinguish); hard negatives (BM25 top non-relevant) force fine distinctions. Modern training mines hard negatives iteratively: retrieve with current model, treat top non-relevant as negatives, retrain (ANCE, RocketQA).

Formal. Contrastive loss (InfoNCE) for query q, positive d^+, negatives $\{d_j^-\}$:

$$\ell = -\log \frac{\exp(s(q,d^+)/\tau)}{\exp(s(q,d^+)/\tau) + \sum_j \exp(s(q,d_j^-)/\tau)}, \tag{8.1}$$

with temperature τ and $s(q,d) = \vec{q}\cdot\vec{d}$ (bi-encoder) or $s(q,d) = Transformer([q;d])$ (cross-encoder). Large batch negatives reuse other queries' positives as negatives (in-batch). Curated negatives plus in-batch gives 100s of negatives per query at low cost.

Remark 8.1 (Evaluation vs. training). Train with contrastive loss but evaluate with NDCG/MAP – a mismatch that listwise losses (LambdaMART, approx NDCG) try to close. Recent work directly optimises NDCG via differentiable sorting. Always report zero-shot (no fine-tune on target collection) as well as fine-tuned, to distinguish memorisation from generalisation.

8.6 From Ranker to System: Indexing, Latency, and Monitoring

Intuition. A neural IR system is not just a model: it is an index (dense + sparse), a retriever (ANN + postings merge), a re-ranker (cross-encoder on top-k), and a monitor (NDCG drift, latency p99). Each layer has its own bottleneck.

Latency budget. Typical budget for web search is <200 ms p99. Dense ANN (5 ms) + sparse BM25 (10 ms) + fusion (1 ms) + cross-encoder re-rank of 50 docs (80 ms on GPU) + feature extraction still fits. Scaling to 10^9 docs means sharding: doc-partitioned ANN and BM25, scatter-gather top-k ($k = 100$) per shard, then global merge – identical to Ch. 4's distributed indexing, now with vectors.

Remark 8.2 (Monitoring). Track query-level NDCG@10 daily (is retrieval quality drifting as corpus grows?) and system NDCG over pinned golden queries (has a model update regressed a known good ranking?). This operational view closes the loop back to Ch. 1's pipeline: crawl → index → model → evaluate → retrain.

8.7 Chapter Notes

LambdaMART (Burges et al., 2010) dominated LTR before deep learning; BERT-based rankers (Nogueira et al., 2019) and dense retrieval (Karpukhin et al., 2020 DPR) shifted the field to neural. The modern consensus is hybrid: sparse (precise, cheap) + dense (semantic, recall) + cross-encoder re-rank (accurate) – mirroring the IR pipeline's candidate-generation then ranking decomposition from Ch. 1.

8.8 Exercises

E8.1 **Loss types.** For query with docs a(rel 2), b(rel 1), c(rel 0) and scores $s_a = 1.0, s_b = 0.6, s_c = 0.7$, compute pointwise squared error, pairwise inversions, and $NDCG@3$. Which loss would penalise the b/c inversion?

E8.2 **Feature design.** List 6 features for (q, d) for query "NTU shuttle bus" (include BM25, PageRank, proximity, and one click feature). Explain how each helps and one way it could be gamed.

E8.3 **Bi vs. cross.** Your corpus has 10^7 docs, QPS 100. Quantify why cross-encoder on all docs is infeasible and propose a two-stage bi-encoder → cross-encoder budget (how many docs to re-rank?).

E8.4 **ANN trade-off.** HNSW with $m = 16$ gives 0.96 recall@10 at 2ms, IVF gives 0.90 at 0.8ms. When would you choose each? How does hybrid sparse+dense fusion affect the required dense recall?

E8.5 **Bias and leakage.** You train on click logs where top-ranked docs get more clicks. Describe position bias and one debiasing method (e.g., IPS). Also describe one train/test leakage check using query or doc overlap.